

Writer of Story, Poetry and Code

My bio has said the same thing for years: *Writer of story, poetry and code*. I wrote that line before I thought too hard about what it meant. I'm thinking hard about it now.

My mother worked as a technical writer in the era when dropping a box of punch cards wasn't a metaphor—it was a catastrophe. The project, the sequence, the *program*—gone, scattered across a linoleum floor. Someone had to get on their hands and knees and reconstruct the order of the universe, card by card. That's not programming. That's liturgy. The machine didn't care about your intentions. It only cared about the sequence.

From there: assembly language. The programmer as translator, speaking in the machine's own tongue—registers, opcodes, memory addresses. A small priesthood, initiated through suffering. Then compilers arrived and the conversation got a little more civilized. Someone built a layer of abstraction between human intention and machine execution, and the priesthood exhaled. Then C. Then C++. Then Java. The dream, barely concealed in each iteration, was always the same: *what if we could just say what we meant?*

[COBOL](#) made a serious attempt in 1959. Business logic in plain English sentences, readable by non-programmers — in theory. That didn't quite work out. But the dream persisted. Ruby arrived and you could write code that *read* like English — or at least like English with very strict grammar and no patience for ambiguity. Python read like pseudocode made real. JavaScript spread everywhere, flawed and beloved, the Latin of the web.

Here's the thing I never found strange about any of this: I only ever thought of language as language. To make it more or less grammatical depending on the subject was always obvious —

even when I was speaking, I was thinking about my audience. Who is listening? Who is reading? The grammar adjusts accordingly.

I write code with strict, very strict grammatical rules. I write English with considerably looser ones. "Green dreams sleep angrily" is a sentence. It means something — you felt it before you analyzed it. Code doesn't allow for this. A compiler rejects it without ceremony. Language holds it, turns it over, finds the light inside it. Poetry lives in exactly that gap between grammatical and meaningful, between what the rules permit and what the mind receives.

The entire history of programming languages is the history of closing that gap — not by loosening code's grammar, but by making the machine climb toward ours.

It has nearly arrived.

Natural language interfaces for coding agents aren't the end of that climb — they're the summit. You describe what you want in plain language. The agent writes the code, runs the tests, catches the errors, refactors the output. And the tools are starting to formalize this. [Speckit](#), GitHub's specification kit, treats the English-language spec as the primary artifact — the thing from which software is derived. Not a document stapled to the back of a pull request. The spec *is* the software, upstream of every line of code that follows.

This has been coming for a long time. Test-driven development — [TDD](#), the practice of writing tests before writing code — was already pointing this direction. You describe the desired behavior first, precisely, in terms the machine can verify. Then the code follows. The description precedes the implementation. The intention leads, and execution catches up.

Git, the version control system every developer lives inside, is a writing discipline in disguise. A good commit message is a sentence that says exactly what changed and why. A pull request is an argument — here is what I did, here is why it is correct, here is what you should check. Code

review is editing. The whole apparatus is language, structured and purposeful, carrying intention from one mind to another across time.

The developer curriculum of the future is a writing curriculum. How to write a product requirements document. How to write a specification. How to write a test before you write the code. How to write a commit message that your collaborator — human or machine — can act on. How to write a prompt that gets you what you actually meant. These are not soft skills orbiting the technical core. They *are* the technical core.

But writing curriculum means something specific. It doesn't mean typing faster or producing more. It means developing the capacity to judge what's in front of you.

Because here's what working with a coding agent actually feels like: you are not making the software. You are art directing it. You describe the shot. You review the take. You say *no, that's not it* — and you redirect. A great art director can walk onto a set and know immediately when the light is wrong without being able to operate the equipment. But they got there by looking at thousands of photographs. By developing an eye. You cannot shortcut that formation.

The same is true here. A developer working with an agent needs taste — the capacity to recognize when the output is technically correct but wrong. When it works but it's brittle. When it passes the tests but it's going to be a nightmare to maintain. When it solves the stated problem but missed the actual one.

And taste is downstream of skepticism. Skepticism is the root. You have to walk up to the output and say *I don't trust this yet*. You have to resist the pull of the plausible. LLM output is very good at being plausible — that's structurally what it is, the most statistically reasonable next token, the smoothest path through the probability space. It will never surprise you the way a real solution surprises you, because surprise requires someone willing to be wrong, to go somewhere

uncomfortable, to follow a problem past the point where it's safe.

The output defaults to anodyne. Smooth, competent, inoffensive, forgettable. It sands off the edges. A developer with taste walks in and says *no* — and knows why.

Skepticism, it turns out, is teachable. Not through being fooled, but through practice — close reading, Socratic questioning, the discipline of asking *what is this actually doing* before accepting what it appears to be doing. Every good writing teacher and every good code reviewer has always been doing this. It just got more urgent.

My bio says *writer of story, poetry and code*. I used to think that was a description of range — look how many modes I can work in. I understand now that it was never three things. It was always one thing, with the grammar dialed to different levels of strictness depending on who was listening. Story for humans with time. Poetry for humans with feeling. Code for machines with no patience for ambiguity whatsoever.

The machine is developing patience. The grammar is loosening at that end of the dial.

Which means the person who knows how to write — precisely, structurally, with intention and taste and earned skepticism — is now the person who can drive the machine at every level. Specification. Prompt. Test. Commit. Story. The skill that was always called soft is now the technical skill. The thing that was always central is now, finally, legible as such.

Writing well has never been more important. We're just now getting around to admitting it.